Lecture 1: MDPs & Dynamic Programming

Leif Döring

June 2, 2026

University of Mannheim

Before we start

Great you came, I very much hope you will enjoy the school!!!

This first lecture is about the mathematical backbone of RL: the basic foundation of discrete stochastic control. We discuss the very core, there are variants and deeper results of all kind!

Today's Plan

From decisions to MDPs

Evaluating a policy

Optimizing a policy

Algorithms

From decisions to MDPs

What is a decision model?

Very high level, we will turn this into Maths.

A decision model is a description of a situation in which an agent has to make a decision (we say take an action) that is supposed to optimize some kind of reward/cost.

Typical examples:

- choose a medical treatment,
- choose an advertisement to display,
- choose a move in a game,
- choose a control input for a robot.

(I) The base setting: two-step experiments and one-step decisions

The simplest stochastic decision problem is based on a simple two-stage stochastic experiment:

- sample an action $A \sim \pi$ from an action set $\mathcal{A}$,
- observe a random outcome from a reward distribution $R \sim P(\cdot \mid a)$.

The distribution of a two-stage experiment is the product law $\pi \otimes P$ of π and P:

$$\mathbb{P}(\text{action } a, \text{reward } r) = (\pi \otimes P)(a, r) := \pi(a) \cdot P(r \mid a).$$

The simplest optimal decision problem

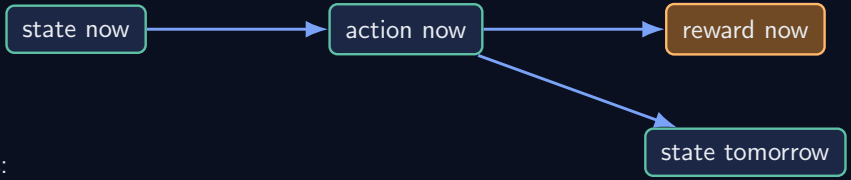
Find an action with maximal expected reward: $a^\star \in \arg\max_{a \in \mathcal{A}} E[R \mid a]$.

A stochastic bandit is a repeated version of this one-step problem:

- each action has an unknown reward distribution,
- by executing decisions the agent learns which actions are good,

(II) Sequential decisions

In many problems, actions have long-term consequences.



Examples:

- a robot moves to a new location,
- a chess move changes the board,
- a treatment changes the future health state of a patient.

Sequential decision making is generally hard

In sequential decision problems, a good action is not necessarily the one with the best immediate reward. Future matters.

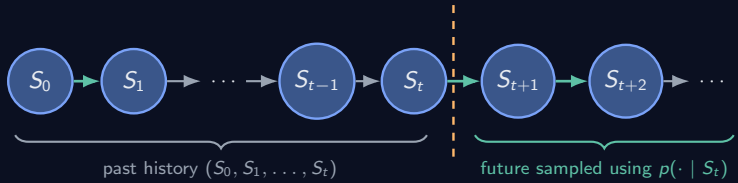
Formal framework: Markov decision models and processes

In order to formalize sequential decision making, a formalized framework is needed. This will consist of

- environments,
- policies,
- Markov decision processes,
- an optimization problem.

To get there we will perform a short excursion through Markov chains and Markov reward chains first.

(I) Markov chains: random dynamics without decisions



Definition: Markov property

A time-homogeneous Markov chain is a stochastic process $(S_t)_{t\geq 0}$ with values in a finite state space $\mathcal{S}$ such that

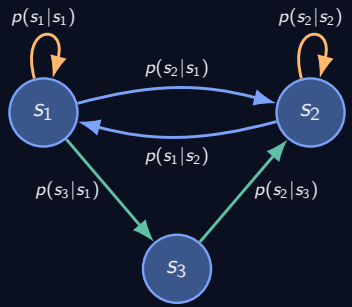
$$\mathbb{P}(S_{t+1} = s' \mid S_0 = s_0, \ldots, S_t = s) = \mathbb{P}(S_{t+1} = s' \mid S_t = s) = \mathbb{P}(S_1 = s' | S_0 = s).$$

The matrix $p(s' \mid s) := \mathbb{P}(S_{t+1} = s' \mid S_t = s)$ is called transition matrix, $\mu(s) = \mathbb{P}(S_0 = s)$ is called initial distribution.

Most of the time we use $\mu = \delta_{s_0}$ for some fixed state s_0, i.e. always start in fixed s_0.

(II) Markov chains: illustration as directed graphs

A finite Markov chain can be visualized as a directed graph.



Edge $s \to s'$ whenever $p(s' \mid s) > 0$.

Limitation

A Markov chain describes how a system evolves, but there is no agent choosing actions.

(III) Markov chains: existence and path probabilities

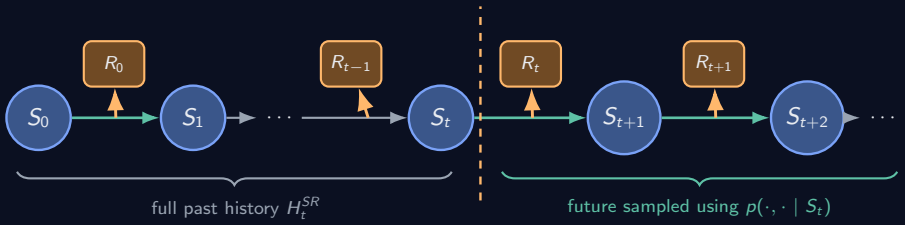
Theorem: Markov chains exist and have nice path probabilities

If p is a probability matrix and μ is a distribution on $\mathcal{S}$, then there is a time-homogeneous Markov chain $(S_0, S_1, \ldots)$ with transition matrix p and initial distribution μ.
Furthermore, for every path of finite length

$$\mathbb{P}(S_0 = s_0, S_1 = s_1, \ldots, S_n = s_n) = \mu(s_0) \prod_{t=0}^{n-1} p(s_{t+1} \mid s_t).$$

multiply the one-step transition probabilities along the path

(IV) Markov reward chain: adding rewards to a Markov chain



Similarly to Markov chains, a Markov reward chain is specified by an initial distribution μ on $\mathcal{S}$ and transition probabilities

$$p(s', r \mid s) = \mathbb{P}(S_{t+1} = s', R_t = r \mid S_t = s).$$

(V) The notion of a kernel

A generalized notion of a stochastic matrix (non-negative entries with rows that sum up to 1) is that of a Markov kernel:

Definition: Markov kernel

For finite sets $\mathcal{X}, \mathcal{Y}$, a mapping $p : \mathcal{X} \times \mathcal{Y} \to [0, 1]$, written $p(y \mid x)$, with

$$\sum_{y \in \mathcal{Y}} p(y \mid x) = 1, \qquad x \in \mathcal{X},$$

is called a Markov kernel from $\mathcal{X}$ to $\mathcal{Y}$.

(VI) Theory of Markov reward chains: existence

Theorem: Existence for a given kernel p and initial distribution μ

Let μ be a probability distribution on $\mathcal{S}$ and let p **be a Markov kernel from** $\mathcal{S}$ **to** $\mathcal{S} \times \mathcal{R}$. Then there exists a stochastic process $(S_0, R_0, S_1, R_1, S_2, \ldots)$ such that $\mathbb{P}(S_0 = s_0) = \mu(s_0)$, and, for every path of finite length,

$$\mathbb{P}(S_0 = s_0, R_0 = r_0, S_1 = s_1, \ldots, R_{n-1} = r_{n-1}, S_n = s_n) = \mu(s_0) \prod_{t=0}^{n-1} p(s_{t+1}, r_t \mid s_t).$$

Theorem: Markov reward property

Moreover, if

$$H_t^{SR} := (S_0 = s_0, R_0 = r_0, \ldots, R_{t-1} = r_{t-1}, S_t = s_t),$$

then the process satisfies the Markov reward property

$$\mathbb{P}(S_{t+1} = s', R_t = r \mid H_t^{SR}) = \mathbb{P}(S_{t+1} = s', R_t = r \mid S_t = s_t) = p(s', r \mid s_t).$$

(VII) Markov decision processes (adding actions)

Definition: Finite decision model (environment)

A finite decision model consists of

$$(\mathcal{S}, (\mathcal{A}_s)_{s\in\mathcal{S}}, \mathcal{R}, p),$$

where

- $\mathcal{S}$ is a finite state space, $\mathcal{R} \subset \mathbb{R}$ is a finite set of rewards,
- $\mathcal{A}_s$ is the finite set of actions available in state s, **from now on** $\mathcal{A}_s = \mathcal{A}$ **for all** s,
- p is a kernel from $\mathcal{S} \times \mathcal{A}$ to $\mathcal{S} \times \mathcal{R}$, we say the transition function.

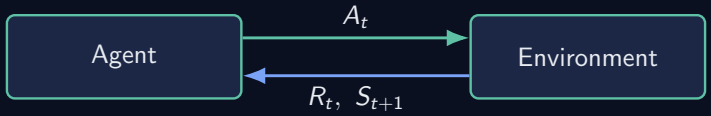
The transition function will turn out to be

$$p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_t = r \mid S_t = s, A_t = a),$$

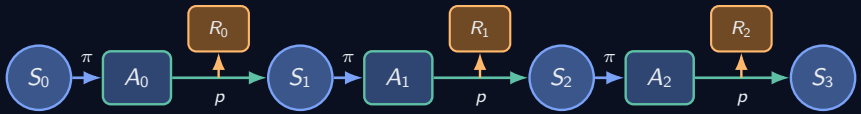
and we abbreviate $p(s' \mid s, a) = \sum_r p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)$.

(VIII) The agent-environment interaction

At every time step, the agent and environment interact as follows.



We still need to specify how the agent chooses actions, once this is done, this is what we get:



(IX) Policies: bringing agents into the game

Definition: (stationary) policy

A kernel π from $\mathcal{S}$ to $\mathcal{A}$ is called a (stationary) policy.

$$\pi(a \mid s) = \text{"probability to choose action } a \text{ in state } s\text{"}$$

In full generality, policies may depend on the whole history

$$h_t = (s_0, a_0, s_1, a_1, \ldots, s_t),$$

so that $\pi_t(a \mid h_t)$ is the probability of choosing action a at time t.

Remark: stationary policies are enough

For discounted return problems stationary policies are sufficient to act optimally. From now on: policy=stationary policy, specify action probabilities $\pi(a \mid s)$ for each state.

(X) Environment + policy = Markov decision process

For environment $(\mathcal{S}, \mathcal{A}, \mathcal{R}, p)$ and policy π we define a kernel q, thus, a Markov reward process:

Definition: Markov decision process

Define the kernel q from $\mathcal{S} \times \mathcal{A}$ to $(\mathcal{S} \times \mathcal{A}) \times \mathcal{R}$ by

$$q\big((s', a'), r \mid (s, a)\big) := p(s', r \mid s, a)\, \pi(a' \mid s').$$

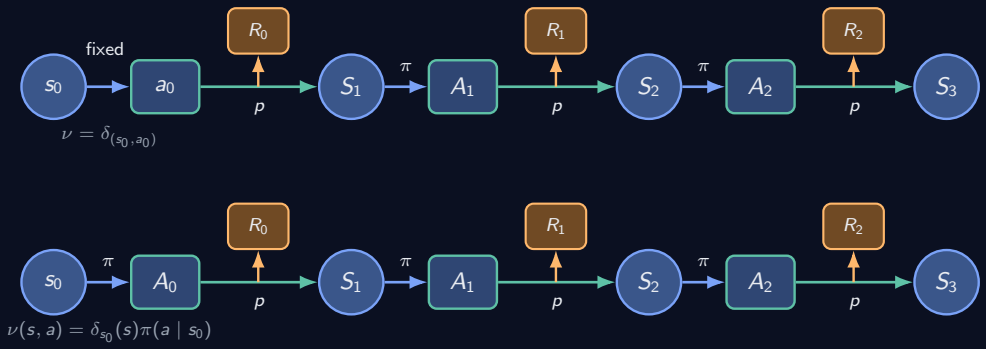
The Markov reward chain $(S_0, A_0, R_0, ...)$ is called **Markov decision process** (from ν, p, π). We will use either $\nu = \delta_{(s_0, a_0)}$ or a fixed initial state with first action sampled from the policy,

$$\nu(s, a) = \delta_{s_0}(s)\pi(a \mid s_0), \qquad \text{i.e. } S_0 = s_0,\ A_0 \sim \pi(\cdot \mid s_0).$$

Why is q a kernel from $\mathcal{S} \times \mathcal{A}$ to $\mathcal{S} \times \mathcal{A} \times \mathcal{R}$?

$$\sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} \sum_{r \in \mathcal{R}} q\big((s', a'), r \mid (s, a)\big) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \sum_{a' \in \mathcal{A}} \pi(a' \mid s') = 1.$$

(XI) Fixed (s, a) vs. fixed s MDP initialization



(XII) Path probabilities and sampling rule

Sampling rule

- sample start $(S_0, A_0) \sim \nu$, often $\nu(s, a) = \mu(s)\pi(a \mid s)$,
- alternate environment p dynamic and policy choice π:
  - reward R_t and next state S_{t+1} from p,
  - action $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$.

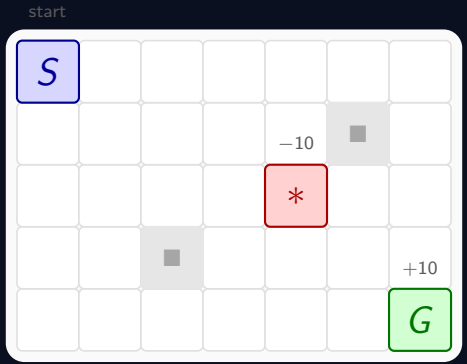
Path probabilities

$$\begin{aligned}&\mathbb{P}_\nu^\pi(S_0 = s_0, A_0 = a_0, R_0 = r_0, \ldots, A_{n-1} = a_{n-1}, R_{n-1} = r_{n-1}, S_n = s_n, A_n = a_n)\\ &= \nu(s_0, a_0) \prod_{t=0}^{n-1} p(s_{t+1}, r_t \mid s_t, a_t)\, \pi(a_{t+1} \mid s_{t+1}).\end{aligned}$$

Example: $\mathbb{P}_{(s,a)}(S_1 = s', A_1 = a') = p(s' \mid s, a)\pi(a' \mid s')$.

Keep the formula in mind: absolutely crucial for policy gradient theorem (p cancel out) and importance sampling (p cancel out) for PPO.

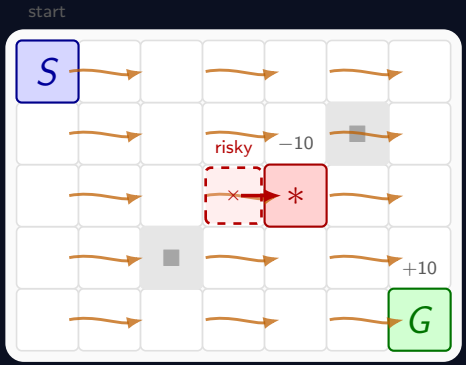
A grid world environment, write down p!



Transitions p are deterministic

The agent starts at S, receives reward $+10$ at the goal G, and reward -10 at the bomb, reward -1 otherwise.

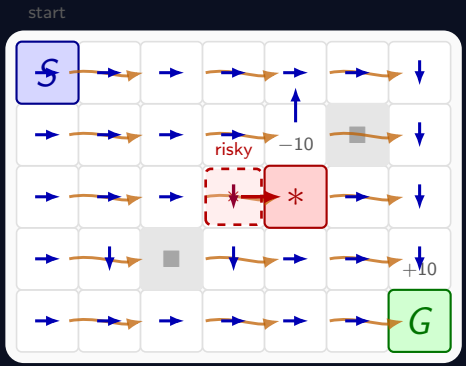
A windy grid world environment, write down p!



Wind makes transitions p stochastic

Shift the intended next step with a small probability to the right.

An exemplary policy environment



rightward wind

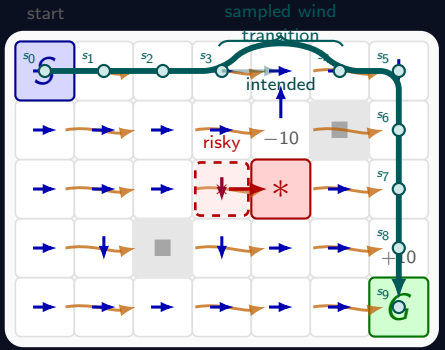
policy π

wind can push into the bomb

Intuition for a good policy (good not defined, yet)

Because wind can push the agent into the bomb, a good policy may take a longer route that keeps distance from dangerous states.

Environment + Policy = Process



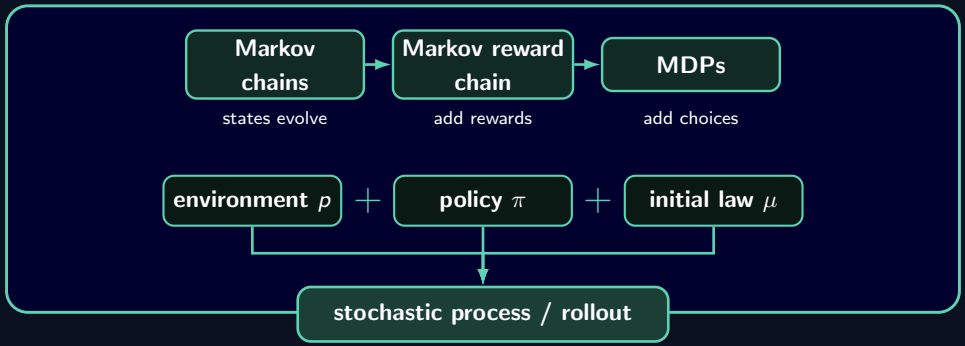
rightward wind

policy π

wind can push into the bomb

a rollout is one sample path

What we have so far



$(S_0, A_0, R_0, S_1, A_1, R_1, \ldots)$ can now be simulated

Next: what makes a policy good and how to find it?

The optimization problem

For an initial distribution μ on S, define the performance of a policy π by

$$J_\mu(\pi) := \mathbb{E}_\mu^\pi\Big[\sum_{t=0}^{\infty} \gamma^t R_t\Big].$$

Discounted MDP problem

Given a finite Markov decision model (S, A, R, p), an initial law μ, and a discount factor $0 \leq \gamma < 1$, find a policy π maximizing expected discounted reward.

Dynamic programming solves this problem and more

Even better: it turns out there is a policy that is optimal for starting states at once.

Evaluating a policy

Value functions and their equations

Notation:

- $\mathbb{E}^{\pi}_{s,a}[\cdot]$ means MDP starts from $S_0 = s$ and $A_0 = a$
- $\mathbb{E}^{\pi}_{s}[\cdot] = \sum_a \pi(a \mid s)\mathbb{E}^{\pi}_{s,a}[\cdot]$ means MDP starts from $S_0 = s$ and $A_0 \sim \pi(\cdot \mid s)$

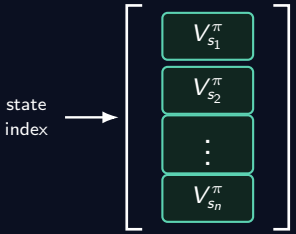
Definition (Value functions of policies):

$$V^{\pi}_s = V^{\pi}(s) := \mathbb{E}^{\pi}_s\Big[\sum_{t=0}^{\infty} \gamma^t R_t\Big], \qquad s \in \mathcal{S},$$

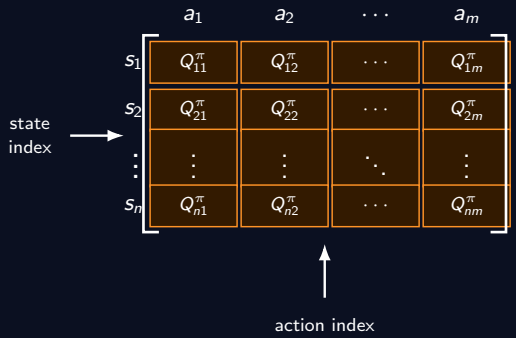
$$Q^{\pi}_{s,a} = Q^{\pi}(s,a) := \mathbb{E}^{\pi}_{s,a}\Big[\sum_{t=0}^{\infty} \gamma^t R_t\Big], \qquad (s,a) \in \mathcal{S} \times \mathcal{A}.$$

State- vs. action-value functions

V^π is a vector



Q^π is a matrix



Keep in mind: V^π is the smaller object!

Bellman expectation equations and relations of V^π and Q^π

Using the notation $r_{s,a} = \mathbb{E}_{s,a}[R_0] = \sum_{s,r} r\, p(s', r \mid s, a)$ the following equations hold:

Theorem: Bellman expectation equations

(i) $$Q^\pi_{s,a} = r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} p(s' \mid s, a)\pi(a' \mid s') Q^\pi_{s',a'}$$

(ii) $V^\pi_s = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q^\pi_{s,a}$

(iii) $Q^\pi_{s,a} = r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) V^\pi_{s'}$

(iv) $$V^\pi_s = \sum_{a \in \mathcal{A}} \pi(a \mid s) \big(r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) V^\pi_{s'} \big)$$

Boxed equations are the core dynamic programming equations, the backbone of optimal control and RL.

Proof of Bellman expectation equation for Q^π: Markov reward property

Step 1: split off the first reward

$$Q^\pi_{s,a} = \mathbb{E}^\pi_{s,a}[R_0] + \mathbb{E}^\pi_{s,a}\left[\gamma R_1 + \gamma^2 R_2 + \gamma^3 R_3 + \cdots\right]$$

Step 2: condition on the next state-action pair

$$\mathbb{E}^\pi_{s,a}\left[R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots\right]$$
$$= \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} \underbrace{\mathbb{P}^\pi_{s,a}(S_1 = s', A_1 = a')}_{\text{known from path probabilities}} \underbrace{\mathbb{E}^\pi_{s,a}\left[R_1 + \gamma R_2 + \cdots \,|\, S_1 = s', A_1 = a'\right]}_{\text{next: Markov reward property}}.$$

Step 3: Path probabilities and Markov reward property

By the Markov reward property, $\mathbb{E}^\pi_{s,a}\left[R_1 + \gamma R_2 + \cdots \,|\, S_1 = s', A_1 = a'\right] = Q^\pi_{s',a'}$, thus,

$$Q^\pi_{s,a} = r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} p(s' \mid s, a)\pi(a' \mid s')Q^\pi_{s',a'}$$

Proof of Bellman expectation equation for V^π and $Q \leftrightarrow V$ transfer

Transfer $Q^\pi \to V^\pi$

$$V_s^\pi = \mathbb{E}_s^\pi\Big[\sum_{t=0}^{\infty}\gamma^t R_t\Big] \overset{\text{Notation}}{=} \sum_{a\in\mathcal{A}}\pi(a \mid s)\,\mathbb{E}_{s,a}^\pi\Big[\sum_{t=0}^{\infty}\gamma^t R_t\Big] = \sum_{a\in\mathcal{A}}\pi(a \mid s)Q_{s,a}^\pi.$$

Transfer $V^\pi \to Q^\pi$

Bellman equation for Q^π and $Q^\pi \to V^\pi$ transfer give

$$Q_{s,a}^\pi = r_{s,a} + \gamma\sum_{s'\in\mathcal{S}}p(s' \mid s,a)V_{s'}^\pi.$$

Bellman equation for V^π

$$V_s^\pi = \sum_{a\in\mathcal{A}}\pi(a \mid s)\Big(r_{s,a} + \gamma\sum_{s'\in\mathcal{S}}p(s' \mid s,a)V_{s'}^\pi\Big).$$

What is "dynamic programming"?

Dynamic programming

Dynamic programming is a method for solving a complex problem by breaking it down into a collection of simpler subproblems.

Recalling the proof of Bellman's expectation equation reveals dynamic programming:

$$\underbrace{Q^{\pi}_{s,a}}_{\text{problem from time 0}} = r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} p(s' \mid s, a)\pi(a' \mid s') \underbrace{Q^{\pi}_{s',a'}}_{\text{problem from time 1}}$$

Since $\infty - 1 = \infty$, the infinite discounted setting does not openly reveal the idea.

Finite-time problems with $Q^{\pi}_{s,a}(k) = \mathbb{E}_{s,a}[\sum_{t=k}^{T} R_t]$ make this clearer:

$$\underbrace{Q^{\pi}_{s,a}(k)}_{\text{problem with } T-k \text{ steps left}} = r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} p(s' \mid s, a)\pi(a' \mid s') \underbrace{Q^{\pi}_{s',a'}(k+1)}_{\text{problem with } T-k-1 \text{ steps left}}$$

Summary: Bellman expectation equations

Bellman expectation equation for V^π

$$V_s^\pi = \sum_{a \in \mathcal{A}} \pi(a \mid s) \Big(r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) V_{s'}^\pi \Big)$$

$$\boxed{V^\pi = r^\pi + \gamma P^\pi V^\pi} \quad \Longleftrightarrow \quad \boxed{V^\pi = (I - \gamma P^\pi)^{-1} r^\pi}$$

Bellman expectation equation for Q^π

$$Q_{s,a}^\pi = r_{s,a} + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q_{s',a'}^\pi$$

$$\boxed{Q^\pi = r + \gamma P_Q^\pi Q^\pi} \quad \Longleftrightarrow \quad \boxed{Q^\pi = (I - \gamma P_Q^\pi)^{-1} r}$$

Bellman expectation equations = linear algebra!

All this is linear algebra. Still need to check that there are unique solutions to Bellman's expectation equations.

Bellman expectation operator

Define two Bellman expectation operator T^π (vector-to-vector and matrix-to-matrix) by

$$(T^\pi V)_s := \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)\,(r + \gamma V_{s'})$$
$$(T^\pi Q)_{s,a} := r(s,a) + \gamma \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} p(s' \mid s, a)\pi(a' \mid s')Q_{s',a'}$$

Then the two Bellman expectation equations become

$$V^\pi = T^\pi V^\pi \quad \text{and} \quad Q^\pi = T^\pi Q^\pi.$$

Contraction property (for Q analogously)

Theorem (T^π are sup-norm contractions):

With the sup-norm $||x||_\infty = \sup |x_i|$, T^π are contraction operators.

Proof:

$$
\begin{aligned}
|(T^\pi V)_s - (T^\pi W)_s| &= \gamma \Big| \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) \left(V_{s'} - W_{s'}\right) \Big| \\
&\leq \gamma \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) \|V - W\|_\infty \\
&= \gamma \|V - W\|_\infty .
\end{aligned}
$$

Taking the maximum over s gives $\|T^\pi V - T^\pi W\|_\infty \leq \gamma \|V - W\|_\infty$.

Policy evaluation theorem

Reminder: Banach fixed-point theorem

If $T : U \to U$ is a contraction on a Banach space $(U, |\cdot|)$, then there is a unique fixed-point $Tu^* = u^*$. Additionally, for all u_0, the iteration $u_{n+1} = Tu_n$ converges in U to u^* with $|u_n - u^*| \leq \gamma^n |u_0 - u^*|$.

Theorem: Policy evaluation

For every stationary policy π there are unique solutions to the Bellman expectation equations (fixed-point equations) and those solutions are the value functions V^π (resp. Q^π).
Those can be computed by matrix inversion or Banach's fixed-point iteration.

Hidden in Banach is the core reason for slow convergence of large γ in RL!

Optimizing a policy

Optimal value functions and their equations

Definition: Optimal value functions

The optimal state-value function is

$$V_s^* := \sup_\pi V_s^\pi, \qquad s \in \mathcal{S}.$$

The optimal action-value function is

$$Q_{s,a}^* := \sup_\pi Q_{s,a}^\pi, \qquad s \in \mathcal{S},\ a \in \mathcal{A}.$$

In analogy to the value functions

- there is a relation between V^* and Q^*,
- there are equations for V^* and Q^*,
- V^* and Q^* are contraction fixed-points.

Unfortunately, this time the equations are non-linear.

Bellman optimality operator/equations

Define the optimality operators as

$$(T^*V)_s := \max_{a \in \mathcal{A}_s} \Big\{ \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)(r + \gamma V_{s'}) \Big\}, \quad s \in \mathcal{S},$$

$$(T^*Q)_{s,a} := \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a)\big(r + \gamma \max_{a' \in \mathcal{A}} Q_{s',a'}\big), \quad s \in \mathcal{S}, a \in \mathcal{A}.$$

The fixed-point equations $T^*V = V$ and $T^*Q = Q$ are called Bellman optimality equations.

Key theorem

(i) T^* are γ-contractions with respect to the sup-norms.
(ii) The unique fixed-points (equivalently, the unique solutions of the Bellman optimality equations) are the optimal value functions V^* and Q^*.

Proof: (i) is easy (similar to T^π), (ii) is more tricky but insightful (we give it for Q^*).

Greedy policy from a matrix: choose the best entry in each row

Q-matrix

	a_1	a_2	a_3	a_4
s_1	1.2	**3.5**	0.7	2.1
s_2	**4.0**	2.6	3.1	1.8
s_3	0.5	1.4	**2.2**	1.9
s_4	2.7	2.7	1.0	**3.3**

take row maximum

greedy policy
chosen action

$\pi(a_2 \mid s_1) = 1$

$\pi(a_1 \mid s_2) = 1$

$\pi(a_3 \mid s_3) = 1$

$\pi(a_4 \mid s_4) = 1$

Key observation: $T^\pi Q = T^* Q$ **for** $\pi = \textbf{greedy}(Q)$

$$\overbrace{r(s,a) + \gamma \sum_{s' \in \mathcal{S}} \sum_{a' \in \mathcal{A}} p(s' \mid s,a)\pi(a' \mid s') Q_{s',a'}}^{(T^\pi Q)_{s,a}} = \overbrace{r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s,a) \max_{a'} Q_{s',a'}}^{(T^* Q)_{s,a}}$$

Proof: why the fixed-point of T^* is Q^*

Let Q^{fix} denote the fixed-point of the optimality operator: $Q^{\text{fix}} = T^* Q^{\text{fix}}$. We show that $Q^{\text{fix}} = Q^*$.

Step 1: Q^{fix} dominates every policy value

For any policy π, the optimality operator dominates the expectation operator:

$$T^* Q \geq T^\pi Q, \qquad \text{for all } Q.$$

This follows from the definition of the operators using $\max_{a' \in \mathcal{A}} Q_{s',a'} \geq \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q_{s',a'}$.
Therefore,

$$Q^{\text{fix}} = T^* Q^{\text{fix}} \geq T^\pi Q^{\text{fix}}.$$

Iterating this inequality and using monotonicity of Bellman operators gives $Q^{\text{fix}} \geq (T^\pi)^n Q^{\text{fix}} \longrightarrow Q^\pi$.
Taking the supremum over all policies,

$$\boxed{Q^{\text{fix}} \geq Q^*.}$$

Proof: why the fixed-point of T^* is Q^*

Step 2: Build a greedy policy from Q^{fix}

Take the greedy stationary policy π^{fix} from the matrix Q^{fix}. For this greedy policy, the maximum in T^* is exactly the action chosen by π^{fix} (key observation above). Hence,

$$T^* Q^{\text{fix}} = T^{\pi^{\text{fix}}} Q^{\text{fix}}.$$

Since $Q^{\text{fix}} = T^* Q^{\text{fix}}$, we obtain $Q^{\text{fix}} = T^{\pi^{\text{fix}}} Q^{\text{fix}}$. Thus Q^{fix} is a also (the unique) fixed-point of the Bellman expectation operator for π^{fix} so that

$$Q^{\text{fix}} = Q^{\pi^{\text{fix}}}.$$

Therefore,

$$\boxed{Q^{\text{fix}} = Q^{\pi^{\text{fix}}} \leq \sup_{\pi} Q^{\pi} = Q^*.}$$

Combining Steps 1 and 2 yields $Q^* = Q^{\pi^{\text{fix}}} = Q^{\text{fix}}$, so Q^* is the fixed-point for T^*.

We actually proved more. The backbone of value based control

Key consequence

The greedy policy π^* of the unique fixed-point Q^* of T^* is optimal in the sense that

$$Q_{s,a}^{\pi^*} = \sup_{\pi} Q_{s,a}^{\pi} \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \quad \left[\text{and also} \quad V_s^{\pi^*} = \sup_{\pi} V_s^{\pi} \quad \forall s \in \mathcal{S} \right]$$

Proof: State-action optimality was part of the previous proof, $\pi^* = \pi^{\text{fix}}$.

The message

Solve fixed-point equation $T^*Q = Q$, then the greedy policy is optimal for all initial states!

Algorithms

What are dynamic programming algorithms?

Simply stated: dynamic programming algorithms are algorithms based on the previous theorems.

Policy evaluation (or prediction) problem

Compute/approximate V^π and/or Q^π.

Optimal control problem

Find an (approximately) optimal policy π^*, i.e. a policy satisfying

$$V_s^{\pi^*} = \sup_\pi V_s^\pi \quad \text{or} \quad Q_{s,a}^{\pi^*} = \sup_\pi Q_{s,a}^\pi.$$

Model-based vs. model-free

In reinforcement learning, the environment is described by transition probabilities $p(s', r \mid s, a)$.

- A method is **model-based** if it assumes access to p.
- A method is **model-free** if it only requires the ability to sample from p.

Think about playing a computer game vs. you wrote the code of the game.

- Dynamic programming is model-based: it computes value functions or policies using a known model.
- Methods like Q-learning, PPO are model-free.

Bellman operators as expectations

$$(T^\pi V)_s = \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} \sum_{r\in\mathcal{R}} p(s', r \mid s, a)\,(r + \gamma V_{s'}) = \mathbb{E}^\pi_s[R_0 + \gamma V_{S_1}]$$

$$(T^\pi Q)_{s,a} = r(s,a) + \gamma \sum_{s'\in\mathcal{S}} \sum_{a'\in\mathcal{A}} p(s' \mid s, a)\pi(a' \mid s')Q_{s',a'} = \mathbb{E}_{s,a}[R_0 + \gamma Q_{S_1,A_1}]$$

$$(T^* V)_s = \max_{a\in\mathcal{A}} \Big\{ r(s,a) + \gamma \sum_{s'\in\mathcal{S}} p(s' \mid s, a) V_{s'} \Big\} = \max_{a\in\mathcal{A}} \mathbb{E}_{s,a}\big[R_0 + \gamma V_{S_1}\big]$$

$$(T^* Q)_{s,a} = \sum_{s'\in\mathcal{S}} \sum_{r\in\mathcal{R}} p(s', r \mid s, a)\big(r + \gamma \max_{a'\in\mathcal{A}} Q_{s',a'}\big) = \mathbb{E}_{s,a}\big[R_0 + \gamma \max_{a'\in\mathcal{A}} Q_{S_1,a'}\big]$$

Approximate dynamic programming

To emphasize how to incorporate sample-based methods in the dynamic programming algorithms we write the Bellman operators as expectations.

Value iteration for policy evaluation (here: V^π, similarly for Q^π)

Goal: compute the value function V^π of a fixed policy π.

Recall the Bellman operator for policy evaluation:

$$(T^\pi V)_s = \mathbb{E}_s^\pi[R_0 + \gamma V_{S_1}].$$

Value iteration for a fixed policy

Choose an arbitrary initial vector V_0, often all 0. Then iterate

$$V^{k+1} = T^\pi V^k,$$

that is,

$$V_s^{k+1} = \mathbb{E}_s^\pi[R_0 + \gamma V_{S_1}^k] \quad \text{for all } s \in \mathcal{S}.$$

This is actually a matrix multiplication. Banach fixed-point iteration: This converges to V^π.

n-step value iteration for policy evaluation ($\to$ TD(λ), GAE)

Goal: compute V^π of a fixed policy π, but using n-step lookahead backups instead of one-step backups.

The n-step Bellman operator is the n-fold composition $(T^\pi)^n = \underbrace{T^\pi \circ T^\pi \circ \cdots \circ T^\pi}_{n \text{ times}}$. Equivalently,

$$\big((T^\pi)^n V\big)_s = \mathbb{E}_s^\pi\Big[\sum_{t=0}^{n-1} \gamma^t R_t + \gamma^n V(S_n)\Big].$$

n-step value iteration for a fixed policy (here: V^π, similarly for Q^π)

Choose an arbitrary initial vector V_0, often all 0. Then iterate

$$V^{k+1} = (T^\pi)^n V^k,$$

that is,

$$V_s^{k+1} = \mathbb{E}_s^\pi\Big[\sum_{t=0}^{n-1} \gamma^t R_t + \gamma^n V^k(S_n)\Big], \qquad s \in \mathcal{S}.$$

This is a Banach fixed-point iteration with contraction factor γ^n. Therefore $V^k \to V^\pi$.

Value iteration for optimal control

Goal: compute the optimal value function Q^*, then play the greedy policy π^*.

Recall the Bellman optimality operator:

$$(T^* Q)_{s,a} = \mathbb{E}_{s,a}\big[R_0 + \gamma \max_{a' \in \mathcal{A}} Q_{S_1,a'}\big].$$

Value iteration for optimal control

Choose an arbitrary initial matrix Q_0, often all 0. Then iterate

$$Q^{k+1} = T^* Q^k,$$

that is,

$$Q^{k+1}_{s,a} = \mathbb{E}_{s,a}\big[R_0 + \gamma \max_{a' \in \mathcal{A}} Q^k_{S_1,a'}\big], \quad s \in \mathcal{S}, a \in \mathcal{A}.$$

Banach fixed-point iteration: This converges to Q^*. Stop at some point and return the greedy policy.

Asynchronous value iteration for optimal control ($\rightarrow$ Q-learning)

Value iteration does not have to update all state-action pairs at every iteration, Gauss-Seidel method.

Instead, at iteration k, choose one pair $(s_k, a_k) \in \mathcal{S} \times \mathcal{A}$ and update only this entry of the matrix Q^k.

Asynchronous value iteration for optimal control

Choose an arbitrary initial matrix Q_0, often all 0. For $k = 0, 1, 2, \ldots$, choose a state-action pair (s_k, a_k) and set

$$Q_{s,a}^{k+1} = \begin{cases} (T^* Q^k)_{s_k, a_k} & : (s, a) = (s_k, a_k) \\ Q_{s,a}^k & : \text{otherwise} \end{cases}.$$

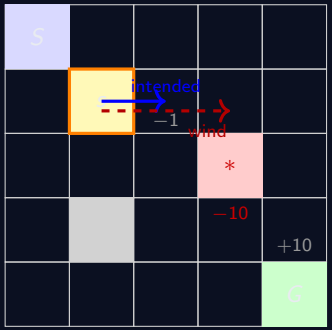
After sufficiently many updates, return the greedy policy.

Convergence condition: every state-action pair $(s, a) \in \mathcal{S} \times \mathcal{A}$ is updated infinitely often.

Towards RL

Asynchronous update are crucial to learn from interactions with the environment!

One Bellman update in the windy grid world, feeling the bootstrap



Current state s, chosen action $a =$ right.

Bellman backup for one state-action pair

$$Q_{s,a}^{k+1} = \sum_{s',r} p(s', r \mid s, a)\Big(r + \gamma \max_{a'} Q_{s',a'}^{k}\Big).$$

Suppose

$$\gamma = 0.9, \qquad p(s_{\text{right}} \mid s, a) = 0.8, \qquad p(s_{\text{wind}} \mid s, a) = 0.2.$$

Also suppose

$$\max_{a'} Q_{s_{\text{right}},a'}^{k} = 4, \qquad \max_{a'} Q_{s_{\text{wind}},a'}^{k} = -6.$$

Then

$$\begin{aligned} Q_{s,\text{right}}^{k+1} &= 0.8 \cdot (-1 + 0.9 \cdot 4) + 0.2 \cdot (-1 + 0.9 \cdot (-6)) \\ &= 0.8 \end{aligned}$$

Bootstrapping through the Q-table (iterations 0,3,6,8)

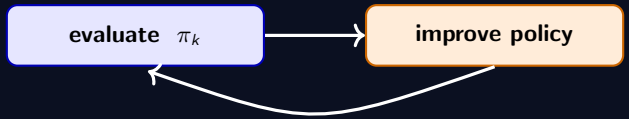
Policy iteration (prototype of actor-critic): evaluate, then improve

Different approach that does not directly use the Bellman equations, but instead involves another trick.

Basic idea

Start with some policy π_0. For $k = 0, 1, 2, \ldots$ repeat:

1. **Policy evaluation:** compute the value function Q^{π_k} of the current policy.
2. **Policy improvement:** choose a new greedy policy $\pi_{k+1} = \text{greedy}\left(Q^{\pi_k}\right)$.



Policy iteration converges

In every step the policy is improved. If it does not improve, then the policy is optimal.

Policy improvement theorem: greedy case using Q

Greedy policy improvement

Suppose π is stationary and $\pi' = \text{greedy}(Q^\pi)$, then π' is an improvement of π:

$$Q^{\pi'}_{s,a} \geq Q^\pi_{s,a} \qquad \text{for all } s \in \mathcal{S},\ a \in \mathcal{A}.$$

Proof: Since π' is greedy with respect to Q^π, we have for every s

$$\sum_{a' \in \mathcal{A}} \pi(a' \mid s) Q^\pi_{s,a'} \leq \max_{a' \in \mathcal{A}} Q^\pi_{s,a'} = \sum_{a' \in \mathcal{A}} \pi'(a' \mid s) Q^\pi_{s,a'}.$$

Therefore, for every (s, a),

$$\begin{aligned} Q^\pi_{s,a} &= r_{s,a} + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s') Q^\pi_{s',a'} \\ &\leq r_{s,a} + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi'(a' \mid s') Q^\pi_{s',a'} = (T^{\pi'} Q^\pi)_{s,a}. \end{aligned}$$

Thus $Q^\pi \leq T^{\pi'} Q^\pi$. By monotonicity of $T^{\pi'}$,

$$Q^\pi \leq T^{\pi'} Q^\pi \leq (T^{\pi'})^2 Q^\pi \leq \cdots \longrightarrow Q^{\pi'}.$$

Bonus material: A quick look into approximate dynamic programming

Approximate dynamic programming has two nested loops. Here for T^*:

- Outer loop: Perform Banach iterations $Q_{n+1} = T^* Q_n$
- Inner loop: Approximate $T^* Q_n$

How to approximate $T^* Q_n$? Using $\mathbb{E}[X] = \arg\min_\theta [(\theta - X)^2]$ write

$$T^* Q(s,a) = \arg\min_{\theta_{s,a}} \mathbb{E}_{r,s'} \Big[\big(\theta_{s,a} - \underbrace{(r + \gamma \max_{a'} Q(s', a'))}_{=:X \text{ "target"}} \big)^2 \Big]$$

and perform SGD with sampled targets:

$$\theta_{s,a}(k+1) = \theta_{s,a}(k) - \alpha_k (\theta_{s,a}(k) - \hat{X}^i_{s,a}) = \theta_{s,a}(k) + \alpha (\hat{X}^i_{s,a} - \theta_{s,a}(k)).$$

What do we get?

If we approximate $T^* Q_n$ with **only one** step SGD in the inner loops and initialize SGD in Q_n (where else?), we get Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big)$$

If we perform SGD with K steps, we get periodic Q-learning (DeepMind's delayed target trick):

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big(r + \gamma \max_{a'} Q^K(s',a') - Q(s,a)\big)$$

Best choice of K in the inner loop? $K_n = \gamma^{-2/3}$! See the poster of Benedikt Wille. [1]

[1] Weissmann et al. "The Role of Target Update Frequencies in Q-Learning", ICML 2026

Enjoy the week!

That's it for now ... if you like to talk about the Maths side of RL, just get in touch!